

# Full Mesh (Consistency Technique)

## 1. Overview

A **Full Mesh** network topology means **every node/host is directly connected to every other node** in the cluster.

Each node knows the addresses of all other nodes and can communicate directly with them.

None

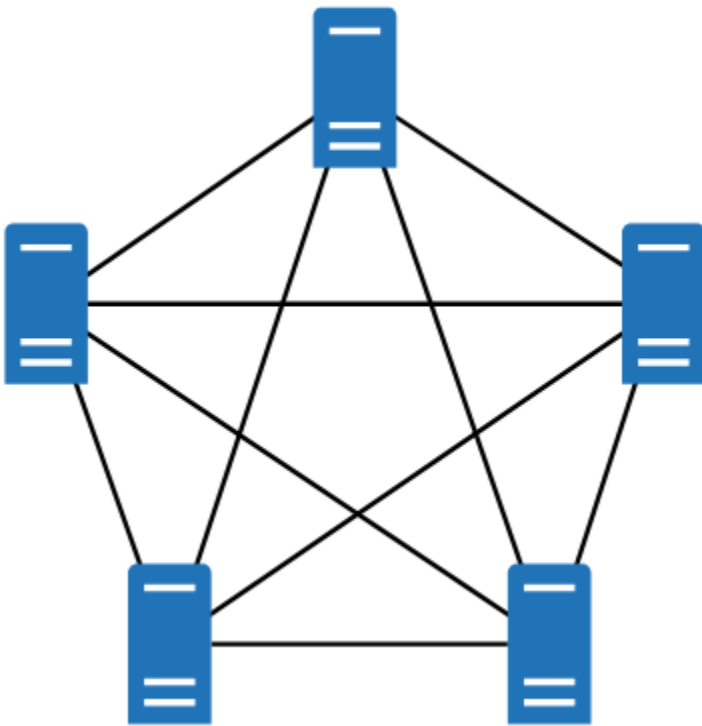
Node A ↔ Node B

Node A ↔ Node C

Node B ↔ Node C

...

Used in distributed systems when all members need fast direct coordination or state sharing.



## 2. Core Idea

If cluster has  $N$  nodes:

Each node maintains connection to:

None

$N - 1$  other nodes

Total connections:

$$\frac{N(N-1)}{2}$$

(For bidirectional pairwise links)

### 3. Example with 4 Nodes

None

A ↔ B

A ↔ C

A ↔ D

B ↔ C

B ↔ D

C ↔ D

Every node can directly send updates to others.

## 4. Why Full Mesh Helps Consistency

Because all nodes talk directly:

- Updates propagate quickly
- No central bottleneck
- Every node can compare state
- Easier immediate synchronization in small clusters

Example:

None

Node A receives write

↓

Broadcast to B, C, D

↓

All nodes update

## 5. Host Discovery Problem

For full mesh to work, nodes must know each other.

Question:

None

How does new node discover all existing nodes?

How do others learn new node address?

## 6. Common Discovery Solutions

### A. Configuration File

Maintain static list:

None

`node1.company.internal`

`node2.company.internal`

`node3.company.internal`

When nodes change:

- Update config
- Redeploy everywhere

### Pros

- Simple

### Cons

- Manual
- Poor for dynamic scaling

## B. Service Registry / Membership Service

Use third-party coordination system:

- ZooKeeper
- etcd
- Consul
- Kubernetes service discovery

Nodes send heartbeats.

As long as heartbeat exists:

None

Node marked alive

All nodes query registry to get member list.

### Pros

- Dynamic
- Automated
- Production-friendly

### Cons

- Extra dependency

## 7. Heartbeat Mechanism

Each node periodically sends:

None

`I am alive`

If heartbeat missing:

None

`Node removed from cluster list`

Used for failure detection.

## 8. Scalability Problem (Very Important)

Full mesh does **not scale well**.

Connections/messages grow quadratically.

If nodes = 10:

$$\frac{10 \cdot 9}{2} = 45$$

If nodes = 100:

$$\frac{100 \cdot 99}{2} = 4950$$

If nodes = 1000:

$$\frac{1000 \cdot 999}{2} = 499500$$

Huge overhead.

## 9. Message Broadcast Cost

If each update is broadcast to all nodes:

None

1 write on 100 nodes

→ 99 outbound messages

High network traffic.

## 10. Resource Costs

Each node must manage:

- Many TCP connections
- Memory for peers
- Heartbeats
- Retry logic
- Encryption sessions
- Membership state

This becomes expensive at scale.

## 11. Where Full Mesh Works Well

### Small Clusters

- 3 to 10 nodes
- Low latency LAN clusters

### Peer-to-Peer Systems

- Small swarms
- Decentralized groups

### Control Plane Clusters

- Small consensus groups

## 12. Example: BitTorrent

BitTorrent uses peer-to-peer communication where peers may connect to many peers directly.

Not always strict full mesh globally, but conceptually direct peer sharing resembles mesh networking.

## 13. Full Mesh vs Leader-Based Systems

| Feature              | Full Mesh       | Leader-Based           |
|----------------------|-----------------|------------------------|
| Central bottleneck   | No              | Yes                    |
| Simplicity (small N) | Good            | Medium                 |
| Scalability          | Poor            | Better                 |
| Coordination traffic | High            | Lower                  |
| Failure of one node  | Often tolerable | Leader election needed |

# 14. Full Mesh vs Quorum

## Full Mesh

Everyone may receive updates directly.

## Quorum

Only majority acknowledgements needed.

None

**N = 5**

**Need 3 nodes ack**

Much more scalable than all-to-all broadcasts.

## **15. Interview Design Insight**

If user asks for 5-node cluster:

Full mesh can be acceptable.

If asks for 500-node distributed database:

Prefer:

- Leader-follower
- Partitioning
- Quorum replication
- Gossip membership

## **16. Real Example Use Cases**

### **Small Database Cluster**

3-node coordination service.

### **Multiplayer Game Region Control**

Small number of servers sync state.

### **Consensus Group**

Raft members communicate heavily (small fixed cluster).

## 17. Failure Handling

If one node dies:

Other nodes detect via heartbeat timeout.

Need to:

- Remove node from peer list
- Reconfigure mesh
- Rebalance traffic

## 18. Interview Answer

**Q: Would you use full mesh for 1000-node cluster?**

Answer:

Usually no. Full mesh causes  $O(N^2)$  connection and messaging overhead. It works well for small clusters, but for large systems I'd prefer partitioning, quorum replication, gossip discovery, or leader-based coordination.

## 19. Best Practices

- Use only for small fixed clusters
- Automate service discovery
- Use heartbeats
- Secure peer communication
- Monitor connection counts
- Replace with scalable topology as cluster grows

## 20. Short Revision Notes

None

Full Mesh = Every node connected to every node

Pros:

- Fast direct sync
- No central bottleneck

Cons:

- $O(N^2)$  connections
- Not scalable

Best for:

Small clusters / control groups